



Software Design Document -- KitchenPal

2025-Autumn-Aberdeen-9

杨靖旖 20233801063

陆美彤 20233801021

Content

1	Introduction	1
2	Requirements Specification	1
2.1	Food Ingredient Management	1
2.2	Expiration Alert System.....	2
2.3	Intelligent Recipe Recommendation.....	2
2.4	Shopping List Management	2
3	Overall Design	2
3.1	User Management Module.....	2
3.2	Ingredient Entry & Management Module	2
3.3	Inventory Management Module.....	3
3.4	Recipe Recommendation & Meal Planning Module.....	3
3.5	Shopping List Module	3
3.6	Expiration Alert Module.....	3
4	User Interface Design.....	3
4.1	Design Principles	3
4.2	Layout & Navigation Design.....	4
5	Key Technologies	4
5.1	Core Programming Language.....	4
5.2	UI Framework & Component Library	4
5.3	Architectural Components & Patterns.....	5
5.4	Navigation Management	5
6	Testing and User Experience Analysis.....	5
6.1	Test Overview	5
6.2	Test Scope and Methodology	6
6.3	Test Results Summary	6

6.4	Key Issues Analysis (Based on User Feedback)	6
7	Conclusion.....	7

1 Introduction

In daily life, many users face problems related to food and diet management: forgetting about ingredients stored in the refrigerator leads to expiration and waste; they struggle with the daily dilemma of "what to eat"; and blind purchasing often results in redundant purchases or missed necessary items. Traditional recipe apps and shopping list apps fail to address these points comprehensively—they lack effective ingredient inventory management, proactive expiration alerts, and recipe recommendations tailored to existing ingredients. To solve these issues, KitchenPal APP is developed as an intelligent solution for refrigerator and diet management.

The core purpose of KitchenPal is to transform "remaining refrigerator ingredients" into "tonight's dining table" seamlessly. By integrating intelligent functions such as ingredient recording, inventory tracking, expiration reminders, recipe recommendations, and shopping list management, the app aims to:

- Minimize food waste caused by expiration or neglect;
- Reduce the decision-making time users spend on "what to eat";
- Optimize the purchasing process to avoid blind buying;
- Provide personalized dietary solutions for different user groups.

KitchenPal focuses on the full lifecycle management of food ingredients and personalized diet matching. Its core function includes:

- Ingredient entry (manual input and barcode scanning);
- Ingredient inventory visualization and expiration date tracking;
- Proactive expiration alerts;
- Intelligent recipe recommendation based on existing inventory and user preferences;
- Shopping list generation and replenishment management.

The app targets mobile device users and will initially support Android systems. Future iterations may extend to health management and cross-platform cooperation, but the current scope is centered on solving core food waste and diet decision-making problems.

2 Requirements Specification

The functional requirements of KitchenPal are derived from core user pain points and value propositions outlined in the product definition, covering end-to-end workflows of ingredient management, diet planning, and waste reduction. All functions shall align with the three core user personas (young office workers, housewives/househusbands, cooking novices) and deliver targeted solutions.

2.1 Food Ingredient Management

- **Dual-Mode Entry:** Support manual input of ingredient details (name, quantity, shelf life) with predefined categories (e.g., vegetables, meats, dairy) for quick selection; provide barcode/QR code scanning functionality to auto-populate product information.

- **Inventory Visualization:** Display ingredients in a grid/list view with color-coded expiration status (green = valid, yellow = 3–7 days left, red = expired/expiring within 2 days).

2.2 Expiration Alert System

- **Proactive Notifications:** Send push notifications for ingredients approaching expiration (default: 3 days prior) and expired ingredients (daily reminder until removed).
- **In-App Alert Center:** Feature a dedicated "Expiring Soon" tab to aggregate all ingredients needing urgent use, with sorting by remaining shelf life.

2.3 Intelligent Recipe Recommendation

- **Inventory-Driven Matching:** Automatically generate recipe suggestions based on available ingredients (prioritizing those near expiration) without requiring manual ingredient selection.
- **Personal-Tailored Optimization:** Provide personalized recipe options based on the individualized needs of different users.

2.4 Shopping List Management

- **One-Click Replenishment:** Let users add missing ingredients from recipe pages to the shopping list with a single tap; auto-populate recommended quantities based on recipe serving sizes.
- **List Organization:** Allow users to edit (add/delete/modify), reorder, and check off items as purchased; categorize shopping list items by store sections (e.g., produce, dairy, frozen foods) to optimize shopping routes.

3 Overall Design

The system is divided into six core modules, each corresponding to key user workflows and UI functionalities outlined in the design specifications. Modules communicate via well-defined interfaces to ensure data consistency and operational efficiency.

3.1 User Management Module

- **Core Functions:** User registration, login (including quick login options), profile management, preference setting (dietary restrictions, notification preferences).
- **Dependencies:** Interacts with the Data Storage Layer to persist user data and the Frontend Layer to render the "Profile Page" and login/registration interfaces.

3.2 Ingredient Entry & Management Module

- **Core Functions:** Support dual-mode ingredient entry (barcode scanning, manual input), ingredient categorization (vegetables, meats, dairy, etc.), and management of ingredient

details (quantity, expiration date, storage location).

- **Dependencies:** Integrates with third-party barcode scanning APIs and product databases; provides data to the Inventory Management Module and Recipe Recommendation Module.

3.3 Inventory Management Module

- **Core Functions:** Visualize ingredient inventory via color-coded status (valid / expiring soon / expired), enable search of ingredients, support sliding operations (edit / delete), and track expiration timelines.
- **Dependencies:** Receives data from the Ingredient Entry Module; triggers the Expiration Alert Module for time-sensitive notifications.

3.4 Recipe Recommendation & Meal Planning Module

- **Core Functions:** Generate inventory-based recipe suggestions, generate personalized recipes based on user requirements, and support meal planning via calendar view.
- **Dependencies:** Retrieves inventory data from the Inventory Management Module; accesses the Recipe Database in the Data Storage Layer; interacts with the Shopping List Module to identify missing ingredients.

3.5 Shopping List Module

- **Core Functions:** One-click addition of missing ingredients (from recipe pages or inventory), categorization of list items (by store sections).
- **Dependencies:** Receives input from the Meal Planning Module and Inventory Management Module; syncs data with the Data Storage Layer for cross-device access.

3.6 Expiration Alert Module

- **Core Functions:** Calculate remaining shelf life of ingredients, send proactive push notifications, and display expiring/expired items in a dedicated in-app tab.
- **Dependencies:** Retrieves expiration data from the Inventory Management Module; integrates with the Frontend Layer to render alert UI and animations.

4 User Interface Design

4.1 Design Principles

KitchenPal's UI design adheres to four core principles derived from the visual and functional requirements outlined in the design specifications:

1. **Simplicity & Clarity:** Minimize cognitive load through clean layouts, concise forms, and intuitive information hierarchy.

2. **Consistency:** Maintain uniform design language (colors, typography, interactions) across all pages to enhance user familiarity.
3. **Efficiency:** Reduce operation steps for core workflows (e.g., ingredient entry, recipe selection) to ≤ 3 taps.
4. **Visual Harmony:** Use a fresh green color scheme and card-based layout to align with the app's "zero-waste" and "healthy living" brand positioning.

4.2 Layout & Navigation Design

All core information (ingredient cards, recipe cards, inventory items) is presented in independent cards, enabling clear information segmentation and reducing visual clutter. Adequate whitespace between cards and UI elements to enhance readability and reduce visual pressure, creating a "breathable" interface.

Core navigation hub with four fixed tabs—"Home", "Inventory", "Meal Plan", and "Profile"—enabling one-tap access to key functional areas. Contextual function buttons (e.g., search, add, filter) on each page to support quick actions (e.g., search ingredients in the Inventory Page, add recipes in the Meal Plan Page).

The page is mainly in green and white colors to convey health, freshness, and environmental friendliness.

5 Key Technologies

5.1 Core Programming Language

The project adopts **Kotlin** as the primary development language, which is the official recommended language for Android development. All core components of the application—including MainActivity, the Screen sealed class, and InventoryViewModel—are implemented using Kotlin, ensuring code reliability, readability, and maintainability.

5.2 UI Framework & Component Library

The UI layer is built on **Jetpack Compose**. The framework simplifies UI development by leveraging composable functions annotated with `@Composable` (e.g., Greeting, GreetingPreview). Built-in Compose components such as Scaffold (for page structure) and Text (for text display) are used to construct interfaces. Additionally, the `@Preview` annotation enables real-time UI preview during development, allowing for rapid iteration and validation of interface designs.

The application integrates the **Material Design 3 (M3)** design system, the latest iteration of Google's design language. Core UI components like Scaffold are sourced from the M3 library, ensuring the interface adheres to modern Android design guidelines. This integration guarantees a consistent, visually cohesive user experience across all pages, aligning with the app's "fresh, user-friendly" design positioning.

5.3 Architectural Components & Patterns

The project follows the **MVVM (Model-View-ViewModel)** architectural pattern to achieve clear separation of concerns:

- **ViewModel:** Components like `InventoryViewModel` manage business logic and data state. They handle asynchronous tasks via `viewModelScope` (a coroutine scope built into `ViewModel`) to ensure efficient data processing without memory leaks.
- **View:** The UI layer is implemented using `Jetpack Compose`. It observes data state exposed by `ViewModel` and updates dynamically, ensuring a unidirectional data flow.
- **Model:** Data access is encapsulated through repositories (e.g., `foodRepository`), which abstract data sources (local databases, APIs) and decouple the UI layer from data management.

The **Jetpack ViewModel** component is used to preserve UI-related data across configuration changes (e.g., screen rotation). By isolating data from the UI layer, `ViewModel` ensures data persistence and prevents redundant data loading. For example, `InventoryViewModel` uses `viewModelScope` to handle background data fetching, eliminating the risk of memory leaks associated with traditional async task handling.

Kotlin Coroutines are employed for non-blocking asynchronous operations. In `InventoryViewModel`, `viewModelScope.launch` initiates coroutines to asynchronously collect data from repositories (e.g., `getAllFoodItems().collect`). This approach avoids UI freezes during data loading, enhancing overall user experience by ensuring smooth interface responsiveness.

5.4 Navigation Management

Page routing is managed using the **Jetpack Navigation** component, which standardizes navigation logic across the app:

- The `Screen` sealed class defines routes for all application pages (e.g., login, home, `recipe_detail/{recipeId}`).
- The `createRoute` method dynamically generates parameterized routes (e.g., passing `recipeId` to navigate to the recipe detail page), simplifying navigation between pages with varying data requirements.
- This component centralizes navigation configuration, reducing coupling between pages and facilitating easy maintenance of the app's navigation flow.

6 Testing and User Experience Analysis

6.1 Test Overview

This section presents the results and analysis of user experience (UX) and compatibility testing for the KitchenPal Android version (v1.0, Beta). The primary objectives of the test were to verify the app's compatibility across different Android devices, evaluate core function usability,

identify interaction bottlenecks, and collect user feedback for iterative optimization.

6.2 Test Scope and Methodology

Regarding software compatibility, we used the Tencent WeTest system to test 8 different brands and various system versions of Android devices to ensure that the test results were representative.

In order to understand the user experience, we invited 12 students with different mobile devices to conduct a software usage task test. The participants completed 5 core workflow tasks, including raw material entry (manual entry/scan), inventory management, recipe browsing, dietary planning, and shopping list editing. Through the post-test survey questionnaire and in-task observations, we collected quantitative data (task completion rate, satisfaction score) as well as qualitative feedback (problem description, improvement suggestions).

6.3 Test Results Summary

In compatibility test, all 8 test devices supported installation, launch, and core function execution without crashes or display abnormalities.

In user experience test, 11 users completed more than 4 core tasks; 2 users failed partial tasks due to unclear interactions. That is to say, the average completion rate for the 12 users was 91.7%. The average user satisfaction score is 4.2, which indicates that users have a high level of satisfaction with the interface design and core value aspects but are dissatisfied with the interaction details.

6.4 Key Issues Analysis (Based on User Feedback)

A. Cumbersome Manual Ingredient Entry Process

- **Issue Description:** Users were required to manually input multiple fields (ingredient name, quantity, expiration date) without quick input options or memory functions. The expiration date entry, in particular, required keyboard input each time, increasing operational friction.
- **User Feedback:** "It's tedious to enter the expiration date with the keyboard every time; it would be better if there were options like 'Tomorrow' or 'Next Week'."

B. Unclear Swipe Operations on Inventory Page

- **Issue Description:** Visual feedback for left-swipe-to-delete and right-swipe-to-edit was insufficient, leading to user confusion and accidental operations.
- **User Feedback:** "I didn't know which direction to swipe for deletion; I had to try several times to figure it out."

C. Weak Calendar Interaction on Meal Plan Page

- **Issue Description:** After clicking a date on the calendar, there was no visual transition when the recommended recipes below updated. Users were unaware of the content

change without deliberate inspection.

- **User Feedback:** "Clicking the date gave no feedback; I had to check the recipes below to realize they'd changed."

D. Missing "Forgot Password" Entry on Login Page

- **Issue Description:** The Login Page only supported account-password login without a password recovery mechanism, posing a risk of account inaccessibility.
- **User Feedback:** "What if I forget my password? There's no way to retrieve it."

7 Conclusion

KitchenPal successfully addresses core food management pain points—expiration waste, meal decision fatigue, and blind purchasing—by delivering a functional Android app integrating ingredient tracking, expiration alerts, personalized recipe recommendations, and shopping list tools.

Built on a modern Android tech stack (Kotlin, Jetpack Compose, MVVM) with modular design, the app ensures maintainability and compliance with Material Design 3 standards. Testing confirms 100% compatibility across 8 Android devices, a 91.7% task completion rate, and 4.2/5 user satisfaction, with minor iterative improvements needed (e.g., streamlined entry, clearer interactions).

Future potential lies in iOS expansion, health-focused feature integration (e.g., nutritional analysis), and cross-platform collaborations. Overall, the project achieves its core goals, providing a user-centric solution with a scalable foundation for growth.